

## MySql&Jdbc 回顾

### 今日内容介绍

- ◆ 回顾 MySQL
- ◆ 回顾 JDBC

### 今日内容学习目标

- ◆ 使用 SQL 语句完成单表的增删改查
- ◆ 使用 JDBC 通过 java 代码完成单表的增删改查

## 第1章 回顾 MySql，完成单表增删改查

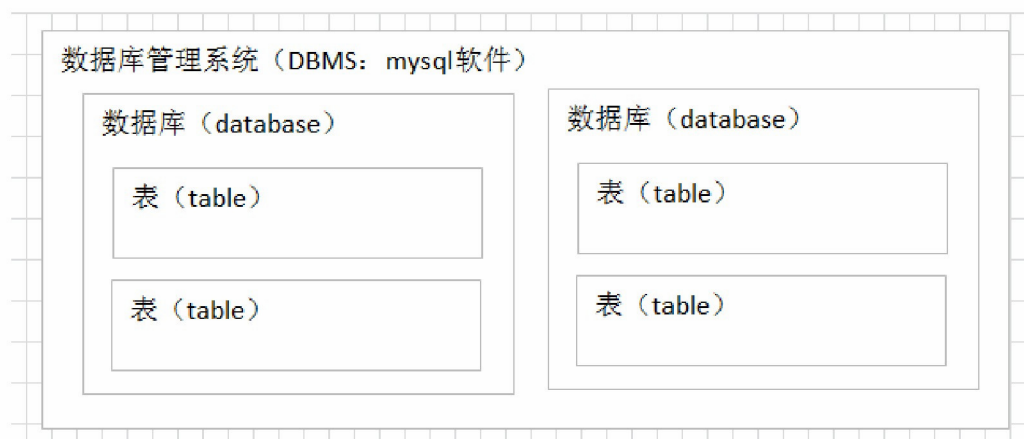
### 1.1 需求分析

在基础班我们已经学习了 SQL 语句，今天我们将在带领大家一起回顾 MySQL 单表的增删改查操作。

### 1.2 数据库回顾

#### 1.2.1 数据库概述

- 什么是数据库  
数据库就是存储数据的仓库，其本质是一个文件系统，数据按照特定的格式将数据存储起来，用户可以通过 SQL 对数据库中的数据进行增加，修改，删除及查询操作。
- 数据库和表



数据库管理系统 (DataBase Management System, DBMS): 指一种操作和管理数据库的大型软件, 用于建立、使用和维护数据库, 对数据库进行统一管理和控制, 以保证数据库的安全性和完整性。用户通过数据库管理系统访问数据库中表内的数据。

- 常见的数据库 (软件)

**MYSQL** : 开源免费的数据库, 小型的数据库. 已经被 Oracle 收购了. MySQL6.x 版本也开始收费。

mysql 变异版: mariadb → 阿里云 centos 内置数据库

**Oracle** : 收费的大型数据库, Oracle 公司的产品。Oracle 收购 SUN 公司, 收购 MYSQL。

**DB2** : IBM 公司的数据库产品, 收费的。常应用在银行系统中。

**SQLServer**: MicroSoft 公司收费的中型的数据库。C#、.net 等语言常使用。

**SyBase** : 已经淡出历史舞台。提供了一个非常专业数据建模的工具 PowerDesigner。

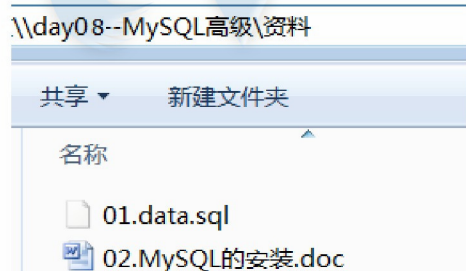
**SQLite** : 嵌入式的小型数据库, 应用在手机端。

Java 相关的数据库: **MYSQL, Oracle**。

## 1.2.2 MySql 使用

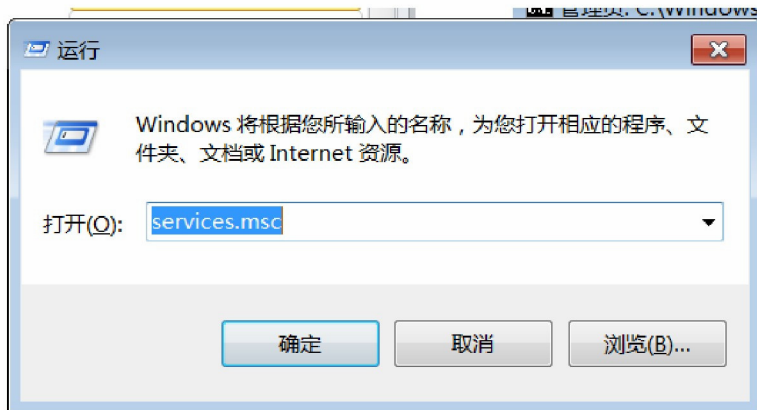
- 安装

参考: 资料/02.MySQL 的安装.doc

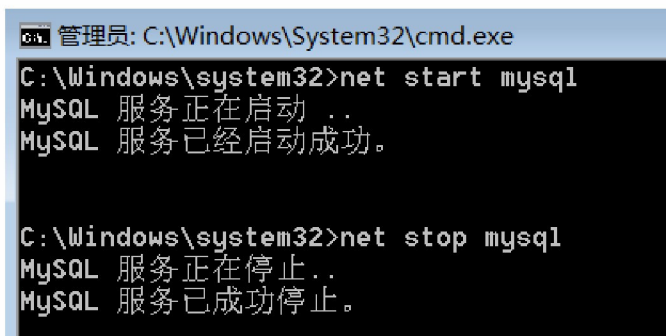


- 打开 window “服务”

■ win + r , “运行” → services.msc



- 启动和关闭（必须以管理运行 cmd 命令窗口）



- 登录

格式 1: cmd> mysql -u 用户名 -p 密码

例如: mysql -uroot -p1234

```
C:\Windows\system32>mysql -uroot -p1234
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 1
Server version: 5.5.27 MySQL Community Server (GPL)
```

格式 2: cmd> mysql --user=用户名--host=ip 地址 --password=密码

### 1.2.3 SQL 语句

- 什么是 SQL 语句

结构化查询语言。关系数据库语言的国际标准。

各个数据库厂商都支持 ISO 的 SQL 标准：普通话

各个数据库厂商在标准基础上做了自己的扩展：方言

- SQL 分类：

- 数据定义语言：简称 DDL(Data Definition Language)，用来定义数据库对象：数据库 database，表 table，列 column 等。关键字：创建 create，修改 alter，删除 drop 等（结构）
- 数据操作语言：简称 DML(Data Manipulation Language)，用来对数据库中表的记录进行更新。关键字：插入 insert，删除 delete，更新 update 等（数据）
- 数据查询语言：简称 DQL(Data Query Language)，用来查询数据库中表的记录。关键字：select，from，where 等

- 数据控制语言：简称 DCL(Data Control Language)，用来定义数据库的访问权限和安全级别，及创建用户；关键字：grant 等

## 1.2.4 数据库操作：database

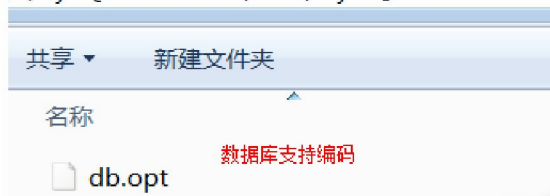
- 创建数据库

语法：

- \* create database 数据库名；
- \* create database 数据库名 character set 字符集；

```
mysql> create database web06_1;  
Query OK, 1 row affected (0.28 sec)  
  
mysql> create database web06_2 character set gbk;  
Query OK, 1 row affected (0.00 sec)
```

L:\MySQL Server 5.5\data\day08\_db



- 查看数据库

查看数据库服务器中的所有数据库：show databases；

查看某个数据库的定义的信息：show create database 数据库名；

- 删除数据库

drop database 数据库名称；

- 其他的数据库操作命令

切换数据库：use 数据库名；

查看正在使用的数据库：select database()；

- 例如：

#创建数据库

```
create database day08_db;
```

#使用数据库

```
use day08_db;
```

## 1.2.5 表操作：table

- 创建表

```
create table 表名(  
    字段名 类型(长度) [约束],  
    字段名 类型(长度) [约束]  
);  
字符类型：varchar(n)
```



单表约束:

- \* 主键约束: primary key, 要求被修饰的字段: 唯一 和 非空
- \* 唯一约束: unique, 要求被修饰的字段: 唯一
- \* 非空约束: not null, 要求被修饰的字段: 非空

#### ● 查看表

查看数据库中的所有表: show tables;

查看表结构: desc 表名;

#### ● 删除表

drop table 表名;

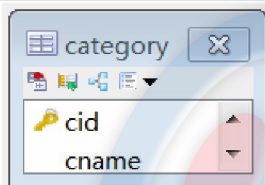
#### ● 修改表

```
alter table 表名 add 列名 类型(长度) [约束];           --修改表添加列.
alter table 表名 modify 列名 类型(长度) 约束;         --修改表修改列的类型长度及约束.
alter table 表名 change 旧列名 新列名 类型(长度) 约束; --修改表修改列名.
alter table 表名 drop 列名;                             --修改表删除列.
rename table 表名 to 新表名;                           --修改表名
alter table 表名 character set 字符集;                 --修改表的字符集
```

#### ● 例如:

###创建分类表

```
create table category(
    cid varchar(32) PRIMARY KEY ,
    cname varchar(100)          #分类名称
);
```



## 1.2.6 字段类型

| 分类   | 类型名称         | 说明                                                                                                                        | 存储需求    |
|------|--------------|---------------------------------------------------------------------------------------------------------------------------|---------|
| 整数类型 | tinyInt      | 很小的整数                                                                                                                     | 1 个字节   |
|      | smallint     | 小的整数                                                                                                                      | 2 个字节   |
|      | mediumint    | 中等大小的整数                                                                                                                   | 3 个字节   |
|      | int(integer) | 普通大小的整数                                                                                                                   | 4 个字节   |
| 小数类型 | float(m,d)   | 单精度浮点数, <ul style="list-style-type: none"><li>● m 表示数字长度</li><li>● d 表示小数位数</li><li>● 例如: float(5,2) 最大值 999.99</li></ul> | 4 个字节   |
|      | double(m,d)  | 双精度浮点数                                                                                                                    | 8 个字节   |
|      | decimal(m,d) | 压缩严格的定点数                                                                                                                  | m+2 个字节 |

|          |              |                                                                    |       |
|----------|--------------|--------------------------------------------------------------------|-------|
| 日期类型     | year         | YYYY 1901~2155                                                     | 1 个字节 |
|          | time         | HH:MM:SS -838:59:59~838:59:59                                      | 3 个字节 |
|          | date         | YYYY-MM-DD 1000-01-01~9999-12-31                                   | 3 个字节 |
|          | datetime     | YYYY-MM-DD HH:MM:SS 1000-01-01 00:00:00~9999-12-31 23:59:59        | 8 个字节 |
|          | timestamp    | YYYY-MM-DD HH:MM:SS 1970~01~01 00:00:01 UTC~2038-01-19 03:14:07UTC | 4 个字节 |
| 文本、二进制类型 | CHAR(M)      | M 为 0~255 之间的整数，占 M 个字节                                            |       |
|          | VARCHAR(M)   | M 为 0~65535 之间的整数，值的长度+1 个字节。占 L+1 个字节。L 小于等于 M                    |       |
|          | TINYBLOB     | 允许长度 0~255 字节，值的长度+1 个字节                                           |       |
|          | BLOB         | 允许长度 0~65535 字节，值的长度+2 个字节                                         |       |
|          | MEDIUMBLOB   | 允许长度 0~167772150 字节，值的长度+3 个字节                                     |       |
|          | LOB          | 允许长度 0~4294967295 字节，值的长度+4 个字节                                    |       |
|          | TINYTEXT     | 允许长度 0~255 字节，值的长度+2 个字节                                           |       |
|          | TEXT         | 允许长度 0~65535 字节，值的长度+2 个字节                                         |       |
|          | MEDIUMTEXT   | 允许长度 0~167772150 字节，值的长度+3 个字节                                     |       |
|          | LONGTEXT     | 允许长度 0~4294967295 字节，值的长度+4 个字节                                    |       |
|          | VARBINARY(M) | 允许长度 0~M 个字节的变长字节字符串，值的长度+1 个字节                                    |       |
|          | BINARY(M)    | 允许长度 0~M 个字节的定长字节字符串                                               |       |

### 1.2.7 插入记录：insert

- 语法：

```
insert into 表 (列名 1,列名 2,列名 3..) values (值 1,值 2,值 3..); -- 向表中插入某些列
insert into 表 values (值 1,值 2,值 3..); --向表中插入所有列
```

- 注意：

1. 列名数与 values 后面的值的个数相等
2. 列的顺序与插入的值得顺序一致
3. 列名的类型与插入的值要一致.
4. 插入值得时候不能超过最大长度.
5. 值如果是字符串或者日期需要加引号'' （一般是单引号）

- 例如：

```
insert into category(cid,cname) values('c001','电器');
insert into category(cid,cname) values('c002','服饰');
insert into category(cid,cname) values('c003','化妆品');
insert into category(cid,cname) values('c004','书籍');
```

| limit                    |        |        |
|--------------------------|--------|--------|
|                          | cid    | cname  |
| <input type="checkbox"/> | c001   | 电器     |
| <input type="checkbox"/> | c002   | 服饰     |
| <input type="checkbox"/> | c003   | 化妆品    |
| <input type="checkbox"/> | c004   | 书籍     |
| *                        | (NULL) | (NULL) |

## 1.2.8 更新记录：update

- 语法：

```
update 表名 set 字段名=值, 字段名=值 ...;
```

```
update 表名 set 字段名=值, 字段名=值 ... where 条件;
```

- 注意：

1. 列名的类型与修改的值要一致。
2. 修改值得时候不能超过最大长度。
3. 值如果是字符串或者日期需要加''。

## 1.2.9 删除记录：delete

- 语法：

```
delete from 表名 [where 条件];
```

- 面试题：

删除表中所有记录使用 `delete from 表名;` 还是用 `truncate table 表名;`

删除方式：delete 一条一条删除，不清空 auto\_increment 记录数。

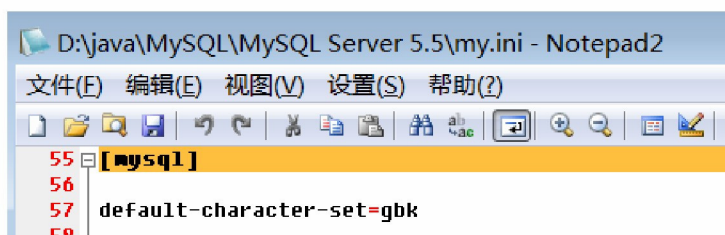
truncate 直接将表删除，重新建表，auto\_increment 将置为零，从新开始。

事务方面：delete 删除的数据，如果在一个事务中可以找回。

truncate 删除的数据找不回来。

## 1.2.10cmd 中文乱码

- 方式 1：修改 my.ini 文件，然后重启 mysql 服务器



- 方式 2：在 cmd 命令行中，当前 cmd 命令窗口有效。

```
cmd> set names gbk;
```

```
mysql> set names gbk;  
Query OK, 0 rows affected (0.00 sec)
```

## 1.3 SQL 查询回顾

查询语句，在开发中使用的次数最多，此处单独回顾，回顾过程使用“product”商品表，为了回顾和巩固，目前使用的表与最终案例表结构略有不同。

- 主键类型，本案例采用 int 类型，实际案例采用是 varchar 类型
- 时间类型，本案例采用 timestamp 类型，实际案例采用 datetime 类型。
- 字段个数，本案例 4 个字段，实际案例采用 7 个字段

### 1.3.1 准备工作

#创建商品表：

```
create table product(  
    pid int primary key auto_increment,  
    pname varchar(20),  
    price double,  
    pdate timestamp  
);
```

#自动增长列：auto\_increment，要求：1.必须整形(int) 2.必须键(主键)

```
insert into product values (null,'谭尼平',0.01,null);  
insert into product values (null,'李士雪',38,null);  
insert into product values (null,'左慈',-998,null);  
insert into product values (null,'黄迎',99999,null);  
insert into product values (null,'南国强',99998,null);  
insert into product values (null,'士兵',1,null);
```

### 1.3.2 语法：

```
select [distinct]* | 列名,列名 from 表 [where 条件].
```

### 1.3.3 简单查询

1. 查询所有的商品. `select * from product;`
2. 查询商品名和商品价格. `select pname,price from product;`
3. 别名查询. 使用的关键字是 as.as 可以省略的.
  - 3.1 表别名: `select * from product as p;`
  - 3.2 列别名: `select pname as pn from product;`
4. 去掉重复值. `select distinct price from product;`
5. 查询结果是表达式 (运算查询): 将所有商品的价格+10 元进行显示.  
`select pname,price+10 from product;`

### 1.3.4 条件查询

1. 查询商品名称为“左慈”的商品所有信息:  
`* select * from product where pname = '左慈';`
  2. 查询商品价格>60 元的所有的商品信息:  
`* select * from product where price > 60;`
- where 后的条件写法:
- \* > , < , = , >= , <= , <>
  - \* like 使用占位符 \_ 和 % \_代表一个字符 %代表任意个字符.  
`* select * from product where pname like '%新%';`
  - \* in 在某个范围中获得值.  
`* select * from product where pid in (2,5,8);`  
`* select * from product where pid=1 or pid = 3 or pid=5;`

|       |                   |                                                                    |
|-------|-------------------|--------------------------------------------------------------------|
| 比较运算符 | > < <= >= = <>    | 大于、小于、大于(小于)等于、不等于                                                 |
|       | BETWEEN ...AND... | 显示在某一区间的值(含头含尾)                                                    |
|       | IN(set)           | 显示在 in 列表中的值, 例: in(100, 200)                                      |
|       | LIKE ‘张 pattern’  | 模糊查询, Like 语句中, % 代表零个或多个任意字符, _ 代表一个字符, 例 first_name like ‘_a%’ ; |
|       | IS NULL           | 判断是否为空                                                             |
| 逻辑运算符 | and               | 多个条件同时成立                                                           |
|       | or                | 多个条件任一成立                                                           |
|       | not               | 不成立, 例: where not(salary>100);                                     |



### 1.3.5 排序

格式: select ... order by 字段 1 asc|desc , 字段 2 asc|desc ...

1. 查询所有的商品, 按价格进行排序. (asc-升序, desc-降序)

```
* select * from product order by price;
```

2. 查询名称有“士”的商品的信息并且按价格降序排序.

```
* select * from product where pname like '%新%' order by price desc;
```

### 1.3.6 聚合

- 常用的聚合函数: sum() 求和 , avg() 平均 , max() 最大值 , min() 最小值, count() 计数;

■ 注意: 聚合函数不统计 null 值

1. 获得所有商品的价格的总和: select sum(price) from product;

2. 获得所有商品的平均价格: select avg(price) from product;

3. 获得所有商品的个数: select count(\*) from product;

### 1.3.7 分组

添加分类 id (cid)

```
alter table product add cid varchar(32);
```

初始化数据

```
update product set cid='1';
```

```
update product set cid='2' where pid in (5,6,7);
```

1. 根据 cid 字段分组, 分组后统计商品的个数.

```
* select cid, count(*) from product group by cid;
```

2. 根据 cid 分组, 分组统计每组商品的平均价格, 并且平均价格 > 60;

```
* select cid, avg(price) from product group by cid having avg(price)>60;
```

### 1.3.8 SQL 语句查询总结

select distinct \* | 字段, ..

from 表

where 查询条件

group by 分组字段

having 分组条件

order by 排序字段 asc|desc

## 第2章 回顾 JDBC，完成查询

### 2.1 案例分析

sql 语句的使用回顾完成，现在我们创建的分类表 `category`，并使用 JDBC 对分类表进行增删改查操作。

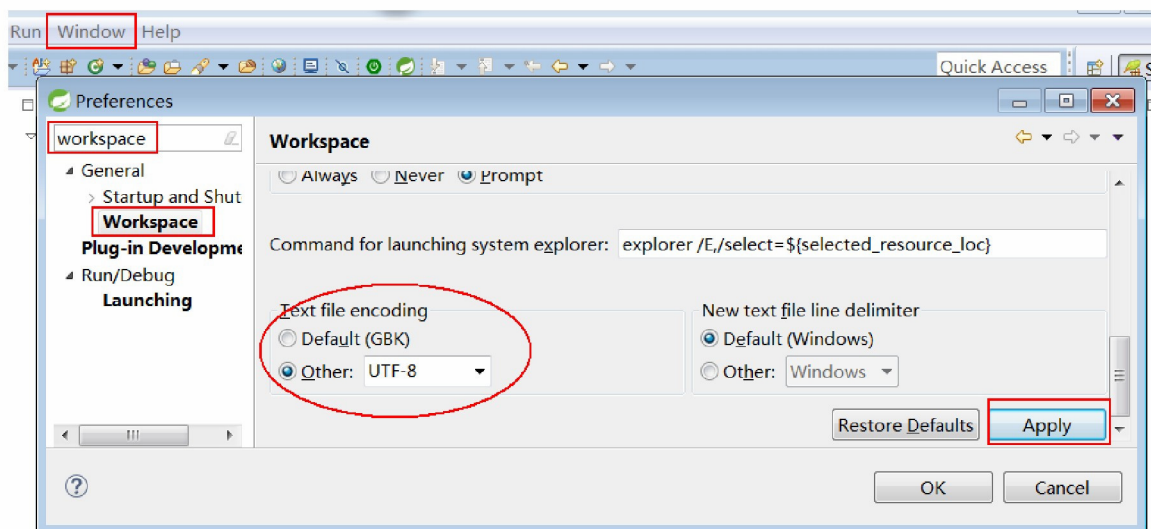
```
###创建分类表
create table category(
    cid varchar(32) PRIMARY KEY ,
    cname varchar(100)          #分类名称
);
#初始化数据
insert into category(cid,cname) values('c001','家电');
insert into category(cid,cname) values('c002','服饰');
insert into category(cid,cname) values('c003','化妆品');
```

### 2.2 案例使用相关技术

#### 2.2.1 eclipse 开发工具

在 Java 就业班将统一使用“eclipse-jee-mars-2-win32”。

- 设置工作空间的编码



## 2.2.2 Junit 单元测试

1. 介绍：JUnit 是一个 Java 语言的单元测试框架。属于第三方工具，一般情况下需要导入 jar 包，不过，多数 Java 开发环境已经集成了 JUnit 作为单元测试工具。  
创建“day07”java 项目，并创建“cn.itcast.a\_junit”包

```
└─ day07
   └─ src
      └─ cn.itcast.a_junit
```

2. 编写测试类，简单理解可以用于取代 java 的 main 方法
3. 在测试类方法上添加注解 @Test
4. 注解修饰的方法要求：public void 方法名() {...}，方法名自定义建议 test 开头，没有参数。

```
DemoAppJUnit.java
1 package cn.itcast.a_junit;
2
3 public class DemoAppJUnit {
4     @Test
5     public void demo01(){
6         System.out.println("测试 demo01");
7     }
8
9 }
10
```

5. 添加 STS 中集成的 Junit 库，鼠标点击“@Test”，使用快捷键“ctrl + 1”，点击“Add Junit ...”

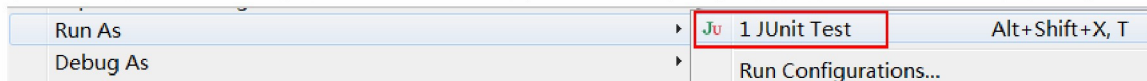
```
3 public class DemoAppJUnit {
4     @Test
5     pub
6
7 }
8
9 }
10
```

Add JUnit 4 library to the build path

结果

```
2
3 import org.junit.Test;
4
5 public class DemoAppJUnit {
6     @Test
7     public void demo01(){
8         System.out.println("测试 demo01");
9     }
10
11 }
```

6. 使用：选中方法右键，执行当前方法；选中类名右键，执行类中所有方法（方法必须标记@Test）



7. 常用注解

@Test，用于修饰需要执行的方法

@Before，测试方法前执行的方法

@After，测试方法后执行的方法

```
public class DemoAppJUnit {
    @Test
    public void demo01(){
        System.out.println("测试 demo01");
    }

    @Before
    public void myBefore(){
        System.out.println("方法前");
    }

    @After
    public void myAfter(){
        System.out.println("方法后");
    }
}
```

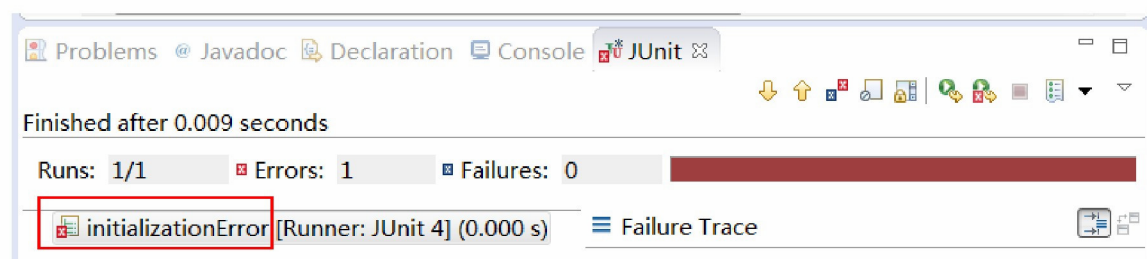
运行结果：

方法前

测试 demo01

方法后

8. 常见使用错误，如果没有添加“@Test”，使用“JUnit Test”进行运行，将抛异常



## 2.3 JDBC 回顾

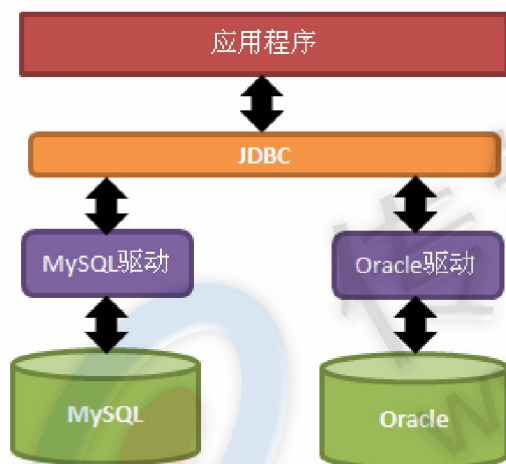
### 2.3.1 JDBC 概述

JDBC, Java DataBase Connectivity: Java 数据库连接, SUN 公司推出的 Java 访问数据库的标准规范 (接口)。

1. JDBC 是一种用于执行 SQL 语句的 Java API。
2. JDBC 可以为多种关系数据库提供统一访问入口。
3. JDBC 由一组 Java 工具类和接口组成。

### 2.3.2 JDBC 原理

SUN 提供访问数据库规范称为 JDBC, 而生产厂商提供规范的实现类称为驱动。



JDBC 是接口, 驱动是接口的实现, 没有驱动将无法完成数据库连接, 从而不能操作数据库! 每个数据库厂商都需要提供自己的驱动, 用来连接自己公司的数据库, 也就是说驱动一般都由数据库生成厂商提供。

当然还有第三方公司专门为某一数据库提供驱动, 这样的驱动往往不是开源免费的!

例如: mysql 驱动包是 `mysql-connector-java-5.1.28-bin.jar`



### 2.3.3 JDBC 开发步骤

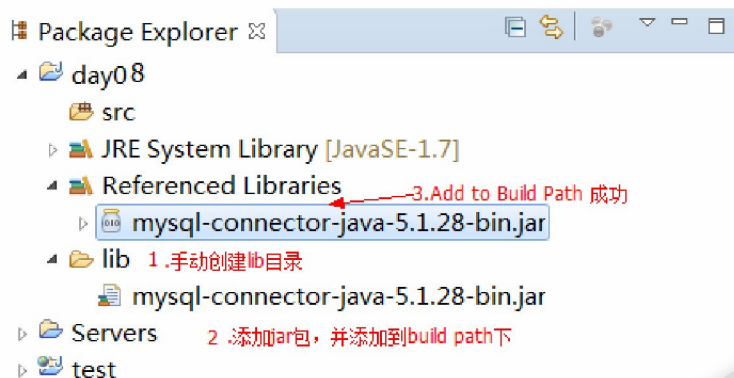
1. 注册驱动.
2. 获得连接.
3. 获得语句执行者
4. 执行 sql 语句
5. 处理结果



## 6. 释放资源.

### 2.3.4 导入驱动 jar 包

1. 创建 lib 目录，用于存放当前项目需要的所有 jar 包
2. 选择 jar 包，右键执行 build path / Add to Build Path



### 2.3.5 API 详解：注册驱动

代码：`Class.forName("com.mysql.jdbc.Driver");`

1. 分析步骤 1: JDBC 规范定义驱动接口: `java.sql.Driver`,  
MySql 驱动包提供了实现类: `com.mysql.jdbc.Driver`
2. 分析步骤 2: `DriverManager` 工具类, 提供注册驱动的方法 `registerDriver()`,  
方法的参数是 `java.sql.Driver`, 所以我们可以通过如下语句进行注册  
`DriverManager.registerDriver(new com.mysql.jdbc.Driver());`  
以上代码不推荐使用, 存在两方面不足
  1. 硬编码, 后期不易于程序扩展和维护
  2. 驱动被注册两次。
3. 分析步骤 3: 通常开发我们使用 `Class.forName()` 加载一个使用字符串描述的驱动类。  
如果使用 `Class.forName()` 将类加载到内存, 该类的静态代码将自动执行。  
通过查询 `com.mysql.jdbc.Driver` 源码, 我们发现 `Driver` 类“主动”将自己进行注册

```
public class Driver extends NonRegisteringDriver implements java.sql.Driver {
    static {
        try {
            java.sql.DriverManager.registerDriver(new Driver());
        } catch (SQLException E) {
            throw new RuntimeException("Can't register driver!");
        }
    }
    .....
}
```

## 2.3.6 API 详解：获得链接

```
代码：Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/mydb1","root","123");
```

获取连接需要方法 `DriverManager.getConnection(url,username,password)`，三个参数分别表示，  
`url` 需要连接数据库的位置（网址） `user` 用户名 `password` 密码

`url` 比较复杂，下面是 `mysql` 的 `url`：

```
jdbc:mysql://localhost:3306/mydb1
```

JDBC 规定 `url` 的格式由三部分组成，每个部分中间使用逗号分隔。

第一部分是 `jdbc`，这是固定的；

第二部分是数据库名称，那么连接 `mysql` 数据库，第二部分当然是 `mysql` 了；

第三部分是由数据库厂商规定的，我们需要了解每个数据库厂商的要求，`mysql` 的第三部分分别由数据库服务器的 IP 地址（`localhost`）、端口号（`3306`），以及 `DATABASE` 名称（`mydb1`）组成。

- 扩展：`url` 参数

```
jdbc:mysql://localhost:3306/mydb1?useUnicode=true&characterEncoding=UTF8
```

`useUnicode` 参数：指定这个连接数据库的过程中，使用的字节集是 `Unicode` 字节集；

`characterEncoding` 参数：指定 `Java` 程序连接数据库的过程中，使用的字节集编码为 `UTF-8` 编码。请注意，`mysql` 中指定 `UTF-8` 编码是给出的是 `UTF8`，而不是 `UTF-8`。要小心了！

## 2.3.7 API 详解：获得语句执行

```
String sql = "insert into category(cid,cname) values('c007','分类')";
```

```
Statement 语句执行者代码：Statement stmt = con.createStatement();
```

常用方法：

- 执行 SQL 语句：

- `int executeUpdate(String sql);` --执行 `insert update delete` 语句。（DML 语句）
- `ResultSet executeQuery(String sql);` --执行 `select` 语句。（DQL 语句）
- `boolean execute(String sql);` --执行 `select` 返回 `true` 执行其他的语句返回 `false`.
  - ◆ 如果返回 `true`，需要使用 `getResultSet()` 获得查询结果
  - ◆ 如果返回 `false`，需要使用 `getUpdateCount()` 获得影响行数

- 执行批处理：（可选）

- `addBatch(String sql);`
- `clearBatch();`
- `executeBatch();`

- 特点：如果有参数，需要在 `sql` 语句中进行拼凑，存在 `sql` 注入问题。

## 2.3.8 API 详解：预处理对象

使用处理对象时，建议每条 sql 语句所有的实际参数，都使用逗号分隔。

```
String sql = "insert into category(cid,cname) values(?,?)";;
```

```
PreparedStatement 预处理对象代码: PreparedStatement psmt = conn.prepareStatement(sql)
```

常用方法:

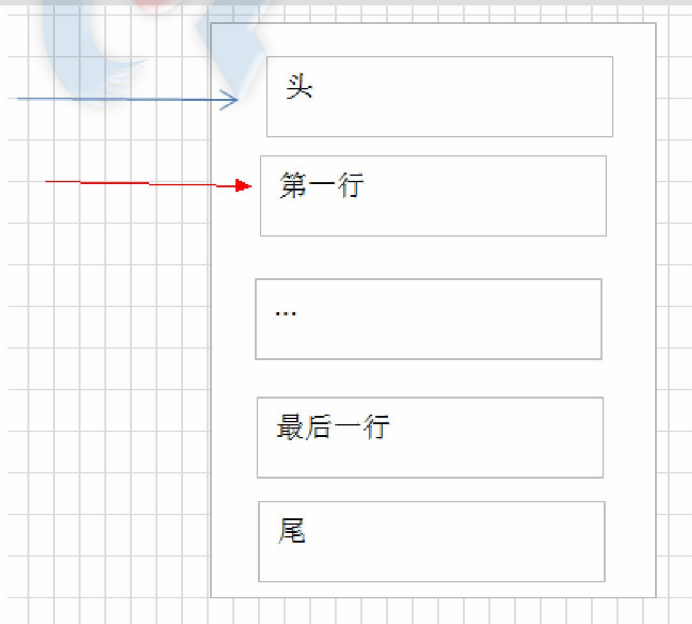
- 执行 SQL 语句:
  - `int executeUpdate();` --执行 insert update delete 语句.
  - `ResultSet executeQuery();` --执行 select 语句.
  - `boolean execute();` --执行 select 返回 true 执行其他的语句返回 false.
- 设置实际参数
  - `setXxx(int, T)` 通过 setter 方法将?占位符替换成实际参数
  - 例如: `setString()` 实际参数类型为 hi 字符串.
- 执行批处理: (可选)
  - `addBatch();` , 添加批处理的实际参数, 需要调方法前执行对应 setter 方法.
  - `clearBatch();`
  - `executeBatch();`

## 2.3.9 API 详解：处理结果集

`ResultSet` 实际上就是一张二维的表格，它内部有一个“行光标”，光标默认的位置在“第一行上方”，我们可以调用 `rs` 对象的 `next()` 方法把“行光标”向下移动一行，当第一次调用 `next()` 方法时，“行光标”就到了第一行记录的位置，这时就可以使用 `ResultSet` 提供的 `getXXX(int col)` 方法来获取指定列的数据了：

```
rs.next(); //光标移动到第一行
```

```
rs.getInt(1); //获取第一行第一列的数据
```



`rs.getInt(1)` 获得数据类型必须是 `int`，如果你不能肯定，那么最好使用 `rs.getObject(1)`。

在 `ResultSet` 类中提供了一系列的 `getXXX()` 方法，比较常用的方法有：

- `Object getObject(int col)`，获得任意对象
- `String getString(int col)`，获得字符串
- `int getInt(int col)`，获得整形
- `double getDouble(int col)`，获得双精度浮点型

### 2.3.10 API 详解：释放资源

与 IO 流一样，使用后的东西都需要关闭！关闭的顺序是先得到的后关闭，后得到的先关闭。

```
rs.close();  
stmt.close();  
con.close();
```

### 2.3.11 回顾案例：查询所有

cn.itcast.b.jdbc\_select

TestSelect.java

```
public class TestSelect {  
  
    //查询所有  
    @Test  
    public void demo01(){  
        try {  
            //1 注册驱动  
            Class.forName("com.mysql.jdbc.Driver");  
            //2 获得连接  
            Connection conn =  
DriverManager.getConnection("jdbc:mysql://localhost:3306/day07_db", "root", "1234");  
            //3 获得语句执行者  
            Statement st = conn.createStatement();  
            //4 执行语句  
            ResultSet rs = st.executeQuery("select * from category");  
            //5 输出结果  
            while(rs.next()){  
                String cid = rs.getString("cid"); //通过【列名】获得值  
                String cname = rs.getString(2); //通过【索引号】获得值，从 1 开始  
  
                System.out.println(cid + " : " + cname);  
            }  
            //6 释放资源  
            rs.close();  
        }  
    }  
}
```

```
        st.close();  
        conn.close();  
    } catch (Exception e) {  
        throw new RuntimeException(e);  
    }  
}  
  
}
```

## 第3章 总结

